ArduPilot Frontend and Backend Pattern

Table of Contents

In ArduPilot firmware, frontend and backend usually mean:

- Frontend: the stable public API used by vehicle code and other libraries.
- Backend: the hardware-specific, protocol-specific, or implementation-specific driver behind that API.

This is common in embedded firmware because the vehicle code should not need to know whether a sensor, actuator, telemetry link, or protocol is implemented by one chip, another chip, SITL, CAN, serial, I2C, SPI, etc.

Pattern Shape

The frontend usually owns:

- public API
- parameters
- shared state
- instance lists

- scheduling/update flow
- detection and backend selection
- common validation and filtering

The backend usually owns:

- hardware-specific reads and writes
- protocol parsing
- bus/device handling
- driver-specific update logic
- pushing measurements or status back into frontend-owned state

This resembles a mix of:

- template method pattern
- strategy pattern
- callback/driver interface pattern

The frontend controls the high-level lifecycle. The backend implements the specific behavior.

Examples in ArduPilot

Common examples include:

- AP_InertialSensor
  - frontend: AP_InertialSensor
  - backend base: AP_InertialSensor_Backend
  - implementations: specific IMU drivers

- AP_RangeFinder
  - frontend: RangeFinder
  - backend base: AP_RangeFinder_Backend
  - implementations: different rangefinder sensors and protocols

- AP_Proximity
  - frontend: AP_Proximity
  - backend base: AP_Proximity_Backend
  - implementations: lidar, MAVLink, DroneCAN, SITL, scripting, etc.

- AP_MSP
  - frontend: AP_MSP
  - backend base: AP_MSP_Telem_Backend
  - implementations: generic MSP, DJI, DisplayPort, etc.

- AP_Networking
  - frontend: AP_Networking
  - backend base: AP_Networking_Backend
  - implementations: networking-specific backends such as PPP

Why ArduPilot Uses This

The frontend/backend split lets vehicle code depend on stable behavior instead of specific devices.

For example, vehicle code can ask the rangefinder frontend for distance without knowing whether the data came from:

- a serial lidar
- an I2C rangefinder
- a DroneCAN sensor
- a MAVLink message
- a SITL simulated sensor

That keeps vehicle logic portable across boards, vehicles, and sensor combinations.

RangeFinder Example

The rangefinder library is a concrete example of the frontend/backend split.

Frontend: RangeFinder

The frontend class is RangeFinder:

- ardupilot/libraries/AP_RangeFinder/AP_RangeFinder.h
- ardupilot/libraries/AP_RangeFinder/AP_RangeFinder.cpp

It defines the stable interface used by vehicle code:

- init(...)
- update()
- has_orientation(...)
- find_instance(...)
- get_backend(...)
- distance_orient(...)
- status_orient(...)
- has_data_orient(...)
- range_valid_count_orient(...)

It also defines the common rangefinder concepts:

- RangeFinder::Type: configured sensor/driver type, such as analog, PWM, MAVLink, DroneCAN, SITL, etc.
- RangeFinder::Status: NotConnected, NoData, OutOfRangeLow, OutOfRangeHigh, Good.
- RangeFinder::Function: conversion function for analog-style sensors.
- RangeFinder_State: normalized sensor state filled by backend drivers.

RangeFinder_State contains frontend-visible measurement state:

- distance_m
- signal_quality_pct
- voltage_mv
- status
- range_valid_count
- last_reading_ms

The frontend owns the arrays that connect all instances together:

```
AP_RangeFinder_Params params[RANGEFINDER_MAX_INSTANCES];
```

```
RangeFinder_State state[RANGEFINDER_MAX_INSTANCES];
AP_RangeFinder_Backend *drivers[RANGEFINDER_MAX_INSTANCES];
```

So the frontend owns parameters, shared state, the list of backend driver pointers, instance count, driver detection, and public getters.

The RangeFinder object itself is owned by AP_Vehicle:

```
#if AP_RANGEFINDER_ENABLED
    RangeFinder rangefinder;
#endif
```

That is why vehicle code can call rangefinder.init(...) and rangefinder.update(...) without constructing the sensor library itself.

Parameters

Vehicle parameter tables expose the rangefinder frontend under a vehicle-specific parameter prefix. In Copter:

```
GOBJECT(rangefinder, "RNGFND", RangeFinder),
```

The frontend then creates per-instance groups:

- RNGFND1_
- RNGFND2_
- ...
- RNGFNDA_

Each instance uses AP_RangeFinder_Params, which defines common parameters such as:

- TYPE
- PIN
- SCALING
- OFFSET
- FUNCTION
- MIN
- MAX
- STOP_PIN
- RMETRIC
- PWRRNG
- GNDCLR
- ADDR
- POS_X, POS_Y, POS_Z
- ORIENT

Backend-specific parameters can also be exposed through backend_var_info when a backend supplies extra parameter metadata.

Backend Base: AP_RangeFinder_Backend

The backend base class is:

- ardupilot/libraries/AP_RangeFinder/AP_RangeFinder_Backend.h

- ardupilot/libraries/AP_RangeFinder/AP_RangeFinder_Backend.cpp

It defines the driver contract:

```
virtual void update() = 0;
```

It also provides common backend helpers:

- distance()
- signal_quality_pct()
- voltage_mv()
- status()
- orientation()
- min_distance()
- max_distance()
- ground_clearance()
- has_data()
- range_valid_count()
- last_reading_ms()

The backend stores references to frontend-owned state and params:

```
RangeFinder::RangeFinder_State &state;
AP_RangeFinder_Params &params;
```

That is the key relationship: the backend does not own the public API state. It writes measurements into the state object that the frontend owns.

The backend base also provides shared status logic:

```
void AP_RangeFinder_Backend::update_status(...) const
{
    if (state_arg.distance_m > max_distance()) {
        set_status(state_arg, RangeFinder::Status::OutOfRangeHigh);
    } else if (state_arg.distance_m < min_distance()) {
        set_status(state_arg, RangeFinder::Status::OutOfRangeLow);
    } else {
        set_status(state_arg, RangeFinder::Status::Good);
    }
}
```

So individual drivers normally read a measurement, store normalized fields, then call update_status().

Concrete Backend Example: Analog Rangefinder

The analog backend is:

- ardupilot/libraries/AP_RangeFinder/AP_RangeFinder_analog.h
- ardupilot/libraries/AP_RangeFinder/AP_RangeFinder_analog.cpp

It inherits from AP_RangeFinder_Backend:

```
class AP_RangeFinder_analog : public AP_RangeFinder_Backend
```

It implements:

- detect(...): checks whether the configured analog pin is usable.
- update(): reads analog voltage, converts voltage to distance, writes frontend state, updates status.

Its runtime flow is:

```
update_voltage();
float v = state.voltage_mv * 0.001f;
state.distance_m = dist_m;
state.last_reading_ms = AP_HAL::millis();
update_status();
```

This is a backend responsibility because it knows how analog voltage maps to a rangefinder distance. The vehicle code never needs to know that conversion.

Lifecycle

For Copter, the lifecycle looks like this:

```
Copter startup
  -> Copter::init_rangefinder()
      -> rangefinder.set_log_rfnd_bit(...)
      -> rangefinder.init(ROTATION_PITCH_270)
          -> RangeFinder::detect_instance(...)
              -> switch on RNGFNDx_TYPE
              -> allocate matching AP_RangeFinder_Backend subclass

Copter scheduler
  -> Copter::read_rangefinder()
      -> rangefinder.update()
          -> for each backend: driver->update()
              -> backend reads sensor/protocol/HAL
              -> backend writes RangeFinder_State
              -> backend updates status
      -> Copter uses frontend state for terrain/surface tracking logic
```

Concrete Copter links:

- ArduCopter/Parameters.cpp: exposes rangefinder as the RNGFND parameter group.
- ArduCopter/system.cpp: calls init_rangefinder() during vehicle startup.
- ArduCopter/sensors.cpp: implements init_rangefinder() and read_rangefinder().
- ArduCopter/Copter.cpp: schedules read_rangefinder.

Other vehicles do the same pattern with different defaults. For example, Plane initializes with ROTATION_PITCH_270, while Rover initializes with ROTATION_NONE.

Relationship Summary

```
Vehicle code
  -> RangeFinder frontend
      -> AP_RangeFinder_Backend base pointer
          -> concrete backend driver
              -> HAL / serial / I2C / CAN / MAVLink / SITL
              -> writes normalized RangeFinder_State
      -> frontend exposes normalized state back to vehicle code
```

Frontend responsibilities:

- stable public API
- common parameters
- instance arrays
- backend detection and allocation
- orientation-based lookup
- common status access
- logging integration

Backend responsibilities:

- specific sensor/protocol implementation
- hardware or message reads
- measurement conversion
- writing RangeFinder_State
- driver-specific status details

In short: vehicle code talks to RangeFinder; RangeFinder chooses and calls an AP_RangeFinder_Backend; the backend talks to the actual sensor path and writes normalized results back into frontend-owned state.

Relation to SRV_Channel

duff/libraries/SRV_Channel is an actuator routing and output abstraction layer.

It maps logical output functions such as:

- throttle
- motor outputs
- aileron
- steering
- lights
- sprayer
- camera trigger
- user actuators

onto physical output channels and output protocols.

SRV_Channel does not follow the same explicit Frontend / Backend naming style as many sensor libraries. Instead, it sits above lower-level output mechanisms such as:

- hal.rcout
- PWM output
- DShot/BLHeli support
- DroneCAN output
- PiccoloCAN output
- SBUS/Volz/Robotis/FETtec output paths, depending on build options

So SRV_Channel is best described as a firmware actuator-output abstraction, while many sensor and protocol libraries are best described as frontend/backend driver frameworks.

Correct Mental Model

For ArduPilot firmware, a useful model is:

- vehicle code uses stable frontend APIs
- frontends manage shared state and policy
- backends implement hardware/protocol-specific behavior
- actuator/output layers route logical commands to physical outputs
- HAL layers perform board-specific IO

Codebase Layout

The duff tree is organized around vehicle firmware, shared libraries, hardware abstraction, tools, and simulation.

Important top-level areas:

- ArduCopter/
  - Copter vehicle application.
  - Contains Copter modes, motor update flow, arming checks, radio handling, GCS vehicle glue, and Copter-specific parameters.
- ArduPlane/
  - Plane vehicle application.
  - Contains Plane modes, fixed-wing servo logic, QuadPlane support, TECS integration, landing logic, and Plane-specific parameters.
- Rover/
  - Rover vehicle application.
  - Contains Rover modes, steering/throttle behavior, sailboat support, wheel encoder integration, and Rover-specific parameters.
- ArduSub/
  - Submarine/ROV vehicle application.
  - Contains Sub modes, joystick handling, lights, camera controls, thruster output, and Sub-specific parameters.
- AntennaTracker/
  - Antenna tracker vehicle application.
  - Contains yaw/pitch tracking logic and tracker servo outputs.
- Blimp/
  - Blimp vehicle application.
  - Contains blimp control, fins, motors, and vehicle parameters.
- Tools/AP_Periph/
  - Peripheral firmware.
  - Builds smaller CAN/peripheral nodes that reuse selected ArduPilot libraries.
- libraries/
  - Shared reusable code.
  - Most of the architecture lives here: sensors, estimators, control libraries, MAVLink/GCS, logging, parameters, HAL, CAN, scripting, terrain, mission handling, motors, actuator outputs, and protocol drivers.
- libraries/AP_HAL/
  - Hardware abstraction interface.

  - Defines abstract hardware services such as scheduler, GPIO, UART, I2C, SPI, storage, RC input, and RC output.

- libraries/AP_HAL_ChibiOS/, AP_HAL_Linux/, AP_HAL_SITL/, AP_HAL_ESP32/
  - Board/platform implementations of the HAL.

- libraries/SITL/
  - Simulation models and simulated devices.

- Tools/autotest/
  - SITL integration test framework.

- modules/
  - External submodules such as MAVLink, ChibiOS, gtest, and other third-party dependencies.

Vehicle Applications

Each vehicle directory is an application built from shared libraries.

Common files and roles:

- *.cpp with the vehicle class
  - Owns the high-level vehicle object, setup flow, loop hooks, and scheduler task table.

- Parameters.h and Parameters.cpp
  - Define vehicle-specific parameter groups and embed shared library parameter groups.
  - For example, vehicle parameters include a SRV_Channels servo_channels object, then expose it under a parameter prefix such as SERVO.

- mode_*.cpp / mode classes
  - Implement vehicle behavior for specific modes.
  - Examples: manual, auto, guided, acro, RTL, loiter, training, etc.

- radio.cpp
  - Reads RC input, configures auxiliary functions, and initializes output functions.

- GCS_*.cpp, GCS_MAVLink_*.cpp, GCS_*.h
  - Vehicle-specific MAVLink/GCS behavior.
  - These files customize generic GCS behavior for each vehicle.

- AP_Arming_*.cpp
  - Vehicle-specific arming checks and arming/disarming side effects.

- wscript
  - Declares libraries required by that vehicle build.

Vehicle code usually should not directly implement low-level device logic. It should call shared libraries through stable APIs.

Scheduler-Driven Execution

Vehicle code is scheduled around periodic tasks rather than one large blocking loop.

The vehicle directories define task tables with macros such as:

- SCHED_TASK

- SCHED_TASK_CLASS
- FAST_TASK

Each task has:

- function or class method
- target rate in Hz
- expected/max runtime budget
- priority/order value

Examples of scheduled work:

- read radio input
- update AHRS
- update GPS
- update barometer
- update current mode
- set servos or motors
- update GCS
- update logging
- run failsafe checks

This is why many library APIs expose an update() method. The vehicle scheduler calls those methods at a controlled rate.

Shared Library Families

The libraries/ directory is not flat conceptually. It contains several recurring families.

Sensor Frontends and Backends

These libraries expose one stable frontend and multiple backends:

- AP_InertialSensor
- AP_GPS
- AP_Baro
- AP_Compass
- AP_RangeFinder
- AP_Proximity
- AP_OpticalFlow
- AP_Airspeed
- AP_BattMonitor
- AP_RPM
- AP_TemperatureSensor

The frontend normally manages:

- parameters
- instance count
- shared state arrays
- health/status checks
- public getters
- backend detection/allocation

- update loop

Backends normally implement:

- sensor detection
- bus reads
- protocol parsing
- sample accumulation
- copying data into frontend state

Estimation and Navigation

These libraries turn sensor data into state estimates and navigation outputs:

- AP_AHRS
- AP_NavEKF2
- AP_NavEKF3
- AP_InertialNav
- AP_GPS
- AP_VisualOdom
- AP_Beacon

The EKF code also uses a frontend/core split. For example, AP_NavEKF3 owns parameters and global selection, while AP_NavEKF3_core instances run the filter math for individual IMU/core lanes.

Control Libraries

These libraries implement reusable controllers:

- AC_PID
- AC_AttitudeControl
- AC_PosControl
- AC_WPNav
- APM_Control
- AP_L1_Control
- AP_Landing
- AR_AttitudeControl
- AR_WPNav

Vehicle code decides which controller to call. The controller libraries calculate desired attitude, rate, steering, throttle, or position outputs.

Motors and Actuator Output

These libraries sit near the output side of the stack:

- AP_Motors
- AR_Motors
- SRV_Channel
- AP_BLHeli
- AP_ESC_Telem
- AP_DroneCAN
- AP_PiccoloCAN

- AP_SBusOut
- AP_RobotisServo
- AP_FETtecOneWire

Typical flow:

```
vehicle mode / controller
  -> motor or servo demand
  -> AP_Motors / SRV_Channels
  -> HAL RCOutput or protocol-specific output
  -> physical actuator
```

SRV_Channel maps logical functions to physical outputs. AP_Motors handles multicopter motor mixing and motor-specific constraints. Protocol libraries handle DShot, CAN, SBUS, and other output mechanisms.

Mission, GCS, Logging, Parameters

These are cross-cutting services used by most vehicles:

- AP_Param
  - Parameter storage, metadata, defaults, conversion, and grouped parameter tables.
- AP_Mission
  - Mission command storage and mission command execution hooks.
- GCS_MAVLink
  - MAVLink message handling and ground-station interface.
- AP_Logger
  - Binary logs, streaming logs, logger backends, and log message definitions.
- AP_Scheduler
  - Cooperative task scheduling and runtime accounting.
- AP_Arming
  - Common arming checks and arming state.
- AP_BoardConfig
  - Board-level configuration and startup checks.

Parameter Architecture

ArduPilot parameters are declared in C++ using AP_Param::GroupInfo tables and macros such as:

- AP_GROUPINFO
- AP_SUBGROUPINFO
- AP_SUBGROUPVARPTR
- GOBJECT
- GGROUP

This gives ArduPilot a consistent parameter system across vehicles and libraries.

Important parameter patterns:

- Vehicle parameter tables include vehicle-owned parameters.

- Shared library parameter tables define library-owned parameters.
- Vehicle code embeds library parameter groups under prefixes.
- Backends can expose backend-specific parameters through backend_var_info.
- Parameter conversion tables preserve old parameter names and formats across firmware upgrades.

For example, a vehicle can expose SRV_Channels as the SERVO parameter group, while each physical channel exposes parameters such as SERVO1_MIN, SERVO1_MAX, SERVO1_TRIM, SERVO1_REVERSED, and SERVO1_FUNCTION.

HAL and Platform Separation

The HAL is the main boundary between portable ArduPilot code and board/platform-specific code.

Portable code typically accesses hardware through:

```
extern const AP_HAL::HAL& hal;
```

Common HAL services include:

- hal.scheduler
- hal.gpio
- hal.uartX / serial manager
- hal.i2c_mgr
- hal.spi
- hal.storage
- hal.rcin
- hal.rcout

Platform-specific implementations live under HAL directories such as:

- AP_HAL_ChibiOS
- AP_HAL_Linux
- AP_HAL_SITL
- AP_HAL_ESP32

This lets the same vehicle and library code run on flight controllers, Linux boards, SITL, and other supported platforms.

Singleton and AP:: Access Pattern

Many core services use a singleton-like pattern:

- class has _singleton
- class provides get_singleton()
- namespace AP provides a convenient accessor

Example shape:

```
namespace AP {
    SomeLibrary &some_library();
}
```

Vehicle and library code then calls APIs such as:

- AP::ahrs()
- AP::logger()
- AP::scheduler()
- AP::srv()
- AP::compass()
- AP::mission()

This makes global services accessible without passing every dependency through every call path. It also means construction order and singleton availability matter.

Compile-Time Feature Flags

ArduPilot uses compile-time configuration heavily.

Common patterns:

- HAL_* flags for board/platform capability.
- AP_*_ENABLED flags for optional library features.
- CONFIG_HAL_BOARD checks for platform-specific behavior.
- HAL_BUILD_AP_PERIPH checks for peripheral firmware builds.

Examples:

- HAL_GCS_ENABLED
- HAL_LOGGING_ENABLED
- HAL_MAX_CAN_PROTOCOL_DRIVERS
- HAL_SUPPORT_RCOUT_SERIAL
- AP_SCRIPTING_ENABLED
- AP_PROXIMITY_ENABLED

This keeps firmware size under control and allows different products to build different subsets of ArduPilot.

Common Architectural Flow

One simplified control cycle looks like this:

```
HAL / backend drivers collect sensor data
  -> sensor frontends publish stable sensor state
  -> AHRS / EKF estimate vehicle state
  -> vehicle mode chooses desired behavior
  -> navigation and control libraries calculate demands
  -> motors / SRV_Channel convert demands to actuator outputs
  -> HAL / protocol backends write physical outputs
  -> logger and GCS report state
```

The important architectural idea is separation of responsibilities:

- vehicle code decides behavior
- libraries provide reusable sensing, estimation, navigation, and control
- frontends hide backend implementation details
- HAL hides board implementation details
- output layers hide actuator routing and protocol details